

METALOGOS

Otcet po 21 Naradam

Implementacija 21 jazykovej fichi v AIML-interpretere
Metalogos: ot arhitektury do bezopasnogo audita

11 ijunja 2026

Z.ai

Soderzanie

- 1.** Vvedenie
- 2.** Svodnaja tablica 21 narada
- 3.** Faza M1-M5: Osnovnye konstrukcii jazyka
- 4.** Faza 1: Sistema tipov
- 5.** Faza 2: Prodvinutye vozmozhnosti
- 6.** Faza 3: Instrumenty razrabotcika
- 7.** Faza 4: Bytecode VM
- 8.** Dopolnitelnye narady (ADR-0045 - ADR-0057)
- 9.** Statistika i metriki
- 10.** Zakljuchenie

1. Vvedenie

Metalogos eto AIML-interpreter, realizovannyj na jazyke Rust. Jazyk prednaznacen dlja sozdanija intellektualnyh prilozhenij, kotorye kombinirujut vyzovy JBesimulirujushih modelej, pravila vyvoda, pamjat s raspadom i adaptivnoe povedenie. Razrabotka vedetsja metodom "contract-first" po postroennoj lestnice milystonov (Build Ladder).

V dannom otchete sistematizirovany vse 21 narada, vypolnennye v ramkah arhitekturnyh reshenij (ADR) s nomerami ot 0001 do 0021. Kazhdyj narad predstavljaet soboj otdelnuju jazykovuju fichu ili podsistemu, proshedsiju polnyj cikl: specifikacija, implementacija, testy, dokumentacija. Otdelno rassmotreny dopolnitelnye narady (ADR-0045 po ADR-0057), vypolnennye v poslednyh sessijah.

Jazyk Metalogos ispolzuet pest 2.x dlja generacii parsera iz PEG-grammatiki, derevovidnyj interpreter dlja ispolnenija AST, trait-abstrakcii dlja LLM/ML backendov i modulnuju arhitekturu (parser, AST, interpreter, compiler, VM, audit). Vse 21 fichi integrirovany v edinyj koda vuzovu s bolee 120 testami, vkluchaja golden-testy, contract-testy i testy semanticheskogo analiza.

2. Svodnaja tablica 21 narada

AD R	Narad	Faz a	Ficha	Kljucevye komponenty
000 1	M1 Arhitektura	M1	Entity, Pattern, Flow	pest PEG, AST, tree-walk interpreter
000 2	M2 Pravila i vetvlenija	M2	Rules, Confidence, Branching	priority-ordered rules, Classify{}
000 3	M3 Learnable Patterns	M3	LLM-integracija	LlmBackend trait, MockLlm, RealLlm
000 4	M4 Pamjat	M4	Memorize, Recall, Forget	Aktivacionnoe raspad, substring recall
000 5	M5 Adapt	M5	Few-shot samomodifikacija	adapt add_example, exact-match cache
000 6	Fluid Types	P1	Tipovaja superpozicija	lazy collapse, COLLAPSE_THRESHOLD
000 7	Confidence Propagation	P1	Rasprostranenie uverennosti	min-konfidents, confidence() builtin

0008	Entity Store	P1	Hranilishhe s identichnostju	type-scoped index, find(), count()
0009	Semantic Analysis	P1	Staticheskaja proverka	two-pass, AnalysisResult, error tests
0010	Codegen / IR	P1	Promezhutochnoe predstavlenie	Program wrapper, semantic validation
0011	Type Inference	P2	Vyvod tipov vyrazhenij	inference rules, error recovery, overlap
0012	Vector Recall	P2	Semanticheskoe podobie	EmbeddingBackend, bag-of-concepts, cosine
0013	ML Backend	P2	Obuchenie patternov	MLBackend trait, learn statement, MockML
0014	Knowledge Graph	P2	Grafovaia pamjat	relate, graph-aware recall, adjacency list
0015	Full Adapt	P2	Mutate, Sandbox, Rollback	mutate, sandbox decl, rollback_if accuracy
0016	CLI + REPL	P3	Instrumenty razrabotcika	clap subcomands, rustyline, mlog check
0017	Stdlib + Import	P3	Standartnaja biblioteka	import std/X, Value::List, map/filter
0018	LSP Server	P3	Integracija v IDE	tower-lsp, diagnostiki, go-to-definition
0019	Package Manager	P3	Upravlenie paketami	mlogpkg, mlog.toml, mlog.lock
0020	Bytecode VM	P4	Stack-VM	30+ opcodes, FlowExec, Collapse
0021	VM Full Coverage	P4	Polnaja paritetnost	strict tests, rule priority, bench 1.28x

Tablica 1. Polnyj spisok 21 narada s ključevymi harakteristikami

3. Faza M1-M5: Osnovnye konstrukcii jazyka

Fazy M1 po M5 sostavljajut fundamentalnyj koren Metalogosa. Kazhdaja faza demonstriruet konceptualnyj proryv: ot prostogo konvejera v M1 do adaptivnogo AI-povedenija v M5. Vse fazy strojatsja posledovatelno, kazhdaja nasleduet

predydushhie i rasshirjaet im.

3.1. M1: Arhitektura i bazovyj cikel (ADR-0001)

M1 dokazal, cikl lexer - parser - AST - interpreter rabotaet skvoz do konca. Dlja parsera byl vybrán pest 2.x (PEG-grammatika), dlja ispolnenija - derevovidnyj interpreter s HashMap hranilishem entity. Kontraktprogramma: **entity, pattern Shout, flow Main**. Osnovnye uroki: pest trebuets imenovannyh pravil dlja operatorov (ne inline-stroki), multi-harakternyj operator "->" trebuets otricatelnogo prosmotra v grammatike dlja razdelenija s "-" (minus).

3.2. M2: Pravila, uverenost, vetvlenija (ADR-0002)

M2 vvvel struct-entity s tipizirovannyimi poljami, pravila s prioriteto (first-wins strategija) i vetvlenie potoka po porogam uverenosti. Pravila sortirujutsja po ubyvaniu prioriteta, pervoe sovpadenie pobezhdaet. Flow-arhitektura razdelena na linejnyj konvejer i bloki vetvlenija: Classify { high (...) -> Escalate, low (...) -> Ignore }. Chetyre vyvodnyh uroka pest: neobhodimost "~" posle povtornyh operatorov, silent-rules razvertyvajut svoi vnutrennij pair, negativnyj prosmotr trebuets osobogo sintaksisa dlja alternativ.

3.3. M3: Learnable Patterns i LLM-integracija (ADR-0003)

M3 - pervyj "vauu-efekt" Metalogosa: learnable pattern otpravljaet prompt LLM i vozvrashaet otvet kak tipizirovannoe znachenie. Realizovana trait-abstrakcija LlmBackend s dvumja implementacijami: MockLlm (deterministicheskij, dlja testov) i RealLlm (HTTP POST cherez curl, podderzhka OpenAI i Ollama formatov). MockLlm pozvoljaet deteministicheskie golden-testy bez setevyh vyzovov.

3.4. M4: Pamjat s raspadom (ADR-0004)

M4 dobavil memorize/recall/forget - hranilishe faktov s aktivacionnym raspadom po formule ACT-R: $\text{activation} = \text{priority} * \exp(-\text{decay} * \text{age_days})$. Recall ispolzuet substring-match dlja poiska i vozvrashaet luchshij rezultat po aktivacii. Pri otsutstvii sovpadenij - soft-failure (pustaja stroka vmesto oshibki). Pamjat in-process, serde-persistence otlozhena.

3.5. M5: Adapt - few-shot samomodifikacija (ADR-0005)

M5 realizuet pervuju formu samomodifikacii: adapt dobavljaet (input, output) pary v few-shot kesh learnable patterna. Pri tochnom sovpadenii vhoda s

keshom LLM ne vyzivaetsja. Bezopasnost garantirovana na urovne tipov: adaptmozno primenjat tolko k learnable patternam, ne k obyknovennym patternam, pravilam ili entity..rollback otlozhen na budushhie fazy.

4. Faza 1: Sistema tipov

Faza 1 vvela tipovuju systemu, semanticheskij analiz i promezhutochnoe predstavlenie. Jeti komponenty neobhodimy dlja nadezhnyh program na Metalogose, prehodyashhih ot prostyh skriptov k slozhnym AIML-prilozhenijam.

4.1. Fluid Types (ADR-0006)

Fluid Types - fundamentalnaja ficha Metalogosa: znachenija, sushestvujushhie v superpozicii konkretnyh tipov s anotacie uverenosti. Deklaracija: `fluid x = Float[42.0][0.9] or String["answer"][0.1]`. Kollaps proishodit lenivo - tolko v tochke ispolzovanija, kogda pattern trebuet konkretnyj tip. Porog kollapsa: `COLLAPSE_THRESHOLD = 0.1`. Pri neudache - soft-failure (`Value::Unit`).

4.2. Confidence Propagation (ADR-0007)

Rasprostranenie uverenosti cherez patterny ispolzuet pravilo min: vyhodnaja uverenost = $\min(\text{urovni uverenosti vseh vhodov})$. Rezultat oborachivaetsja v `Value::Fluid` s odnim variantom, nesushim propagirovannuju uverenost. Builtin `confidence(v)` izvlekaet uverenost iz Fluid znachenija ili 1.0 dlja konkretnyh znachenij. Eto evristika, ne verojatnostnyj vyvod - prostaja i zashishimaja.

4.3. Entity Store (ADR-0008)

Entity Store - tipizirovannyj indeks v `HashMap` peremennyh. Kazhdyj `struct-entity` avtomaticheski registriruetsja v `entity_store: HashMap<String, Vec<EntityRecord>>`. Novye builtin: `find(type, field, op, threshold)` - linejnoe skanirovanie s pervym sovpadeniem, i `count(type)` - kolichestvo entity dannogo tipa. Store - indeks, ne kopija: dejstvitelnye znachenija ostaetsja v variables.

4.4. Semantic Analysis (ADR-0009)

Dvuhprohodnyj analiz pered ispolneniem: prohod 1 sobiraet vse opredelenija tipov i signatury, prohod 2 validiruet ssylki. Proverjaet: neopredelennye entity, neizvestnye shagi potoka, nepozdannye funkcii, tseli pravil, tseli adapt. Rezhim "fail fast" - pervaja oshibka prekrashhaet analiz. Error-testy ispolzujut konvenciju: `err_*.mlog + err_*.error`.

4.5. Codegen / IR (ADR-0010)

Промежхотоchnoe predstavlenie v Faze 1 - tonkij Program-wrapper, obernuvshej AST posle semanticheskogo analiza. Codegen::compile() vypolnjaet semantic::analyze() i sozdaet Program { declarations }. Eto ustanavlivaet arhitektornuju granicu dlja budushej bytecode-VM. Pipeline: source - parse - semantic - codegen (wrap) - interpreter.

4.6. Type Inference (ADR-0011)

Dopolnenie semanticheskogo analiza: vyvod tipov vyrazhenij, vosstanovlenie posle oshibok (sborochnaja AnalysisResult s errors + warnings), obnaruzhenie peresekajushhihsja vetvlenij. Vывod tipov napravlenym rasprostraneniem cherez potok: tip istocnika - proverka parametra shaga - tip vozvrata. Interpoljacija vetvlenij cherez intervalnuju arifmetiku na chislovoj osi.

5. Faza 2: Prodvintye vozmozhnosti

Faza 2 dobavila vektornoe vosstanovlenie pamjati, ML-obuchenie patternov, grafoviju pamjat i polnuju sistemu adaptacii s rollback. Jeti fichi prevrashhajut Metalogos iz prostogo LLM-obertki v platformu dlja adapivnyh AIML-sistem.

5.1. Vector Recall (ADR-0012)

EmbeddingBackend trait abstragiruetsya istochnik vektornykh predstavlenij. Faza 2.2: SimpleEmbeddingBackend s bag-of-concepts (8 grup, ~300 slov). Algoritm: tokenizacija - otozhestvlenie slov s koncept-gruppami - postroenie 8-mernogo vektora - normalizacija - cosine similarity. Rezultat: recall("food preferences") nakhodit "user likes spicy food" bez obshih slov cherez food_cuisine + preference_opinion.

5.2. ML Backend (ADR-0013)

MIBackend trait i operator learn Name with { data, epochs }. MockMIBackend vozvrashaet accuracy=0.95 dlja deterministicheskikh testov. Podderzhka INT literalov v grammatike (dlja epochs: 5). Put prodvinutogo obuchenija: PyO3MIBackend s PyO3+PyTorch, eksport v ONNX dlja runtime.

5.3. Knowledge Graph (ADR-0014)

Novyj operator relate "A" to "B" as "coworker" sozdaet rebra v adjacency list (Vec<Relation>). Pri recall graph walk nakhodit svjazannye fakty i dobavljaet ih kak [GRAPH] relation -> value. Ogranichenija: string-match, netranzitivnyj obhod, linejnoe skanirovanie. Posle 3.0: Neo4j migracija.

5.4. Full Adapt (ADR-0015)

Tri novyh konstrukcii: sandbox (konfiguracija bezopasnosti), mutate (zamena few-shot seta s proverkoj accuracy i rollback), sandbox_decl (allowed/forbidden/timeout). Raznica adapt vs mutate: adapt dobavljaet primery, mutate zamenjaet ves set. Mock accuracy 0.95 pozvoljaet testirovat oba puti (kept i rolled back). Sandbox ne prinuzhdaetsja v runtime - otlozheno na Fazu 3.

6. Faza 3: Instrumenty razrabotcika

Faza 3: CLI, REPL, standartnaja biblioteka, LSP-server i paketnyj menedzher. Jeti komponenty prevrashajut Metalogos iz eksperimentalnogo prototipa v polnocennyj jazyk razrabotki s IDE-podderzhkoj.

6.1. CLI + REPL (ADR-0016)

Rasshiren CLI: mlog run, mlog repl, mlog check. REPL na rustyline s postojannym sostojaniem interpretera mezhdu vvodami. mlog check vypolnjaet semanticheskij analiz bez ispolnenija. feed_line() - public API dlja instrumentov, LSP i notebook-ov. TTY-detection cherez libc::isatty. Istori REPL: ~/.mlog_history.

6.2. Standartnaja biblioteka (ADR-0017)

Import-mehanizm: import std/string razreshaetsja v <std_root>/std/string.mlog. Tri modulja: std/string (trim, replace, split, join), std/math (abs, min, max, clamp, round), std/collections (first, last, push + map/filter/reduce). Dvuhslojnoj pattern-wrapper: Rust builtins s prefiksom "__" + .mlog patterns dlja public API. Value::List tip.

6.3. LSP Server (ADR-0018)

Otdelnyj krate mlog-lsp na tower-lsp. Vozmozhnosti: diagnostiki, go-to-definition, hover, completion, text sync (FULL). Span-propagation v AST dlja pozicij. AnalysisResultDetailed s diagnostikami. Text-scanning podhod dlja go-to-definition i hover (ne trebuetsja spanov na urovne vyrazhenij). VS Code extension manifest.

6.4. Package Manager mlogpkg (ADR-0019)

Otdelnyj binarnyj krate mlogpkg s komandami: init, add, build, info. Manifest: mlog.toml (name, version, edition, dependencies). Lokalnyj registr: ~/.mlog/registry/. Lock-file: mlog.lock. Build workflow: read manifest - resolve

deps - check sources - write lock. Net udistalennogo registr - paketyustanavlivajutsja vruchnuju.

7. Faza 4: Bytecode VM

Faza 4 vvela bytecode-kompiljator i stack-based VM, obespechivaja otdelenie kompiljacii ot ispolnenija i sozdaja bazu dlja optimizacij.

7.1. Bytecode VM (ADR-0020)

Tri novyh modulja: bytecode.rs (Instruction enum), compiler.rs (AST -> Program), vm.rs (stack-based execution). 30+ opcodeov, vklyuchaja Metalogos-specific: JumpIfLow (confidence branching), Collapse (fluid to type), Memorize/Recall/Forget, Adapt/Relate/Mutate, FlowExec, ExecuteRules. Dvuhprohodnaja kompiljacija: pass 1 - sobiranie tipov, pass 2 - generacija instrukcij. Polnaja paritetnost so vseimi 11 golden-test primjerami.

7.2. VM Full Coverage (ADR-0021)

Ispravleny semanticheskie razryvy: pravilnoe sortirovanie pravil po prioritetu, propagacija collections_loaded flaga, strogie dvuhrezhimnye testy. Proizvoditelnost: 1.1x pri 10 shagah, 1.28x pri 1000 shagah, 1.17x srednee na golden-testah. Skromnoe uskorenie objasnjaetsja obshim Value tipom s clone() i makro-instrukcijami (FlowExec, ExecuteRules).

8. Dopolnitelnye narady (ADR-0045 - ADR-0057)

Posle osnovnyh 21 fich byli vypolneny dopolnitelnye narady, rasshirjajushhie vozmozhnosti jazyka dlja serioznyh AIMA-prilozhenij. Nizhe predstavleny kljuchevye iz nih.

8.1. Hooks: before_pattern / after_pattern (ADR-0045)

Perhvatki vyzovov patternov dlja cross-cutting concerns: logging, metriki, audit. Dva vida: before_pattern i after_pattern. V inekcirovannyh peremennyh: pattern_name, args, result, confidence. 6 contract-testov, vse projdeny.

8.2. Context Auto-Loading (ADR-0046)

Avtomaticheskaja podgruzka konteksta dlja learnable patternov. Chetyre rezhima: context: auto (recall po pervomu parametru), context: none, context: recall(expr, limit=N), context: "literal string". Kontekst vstavljatsja pered system prompt v formate "Relevant context: - fact1, - fact2".

8.3. Session Memory (ADR-0049)

Vremennaja pamjat razgovora: session_set, session_get, session_clear. In-memory HashMap s izolacijej po session_id. Net persistence - dannye terjajutsja pri restarte. 10 contract-testov.

8.4. Eval Harness (ADR-0050)

Avtomaticheskaja ocenka learnable patternov na pomeshennyh dannom. Konstrukt eval Name { dataset, metric, threshold }. Confusion matrix, adapt-suggestii dlja fail-primjerov. CLI: mlog eval. Exit code 0/1 dlja CI. 9 contract-testov.

8.5. inspect() Builtin (ADR-0051)

Builtin dlja nabljudenija: inspect("PatternName") -> Struct s 8 poljami. Vozvrashaet: calls, avg_confidence, cache_hits, cache_misses, last_adapt, last_call, examples_count, is_learnable. In-memory, Mutex-protected HashMap. 8 contract-testov.

8.6. Event Stream (ADR-0052)

Edinyj strukturivannyj log vseh operacij interpretera. Event { id, timestamp, event_type, source, data, duration_ms }. Instrumentirovany: memory_store, adapt, pattern_call. 4 builtin: event_count, events_since, event_sum. 9 contract-testov.

8.7. Conversation State (ADR-0053)

Upravljaemyj kontekst dialoga: conv_start, conv_add, conv_history, conv_context, conv_end. Avtoszhatie staryh soobshhenij cherez LLM. Ogranichenie max_messages s evikciej starshego. Integracija s learnable pattern cherez pole conversation. 10 contract-testov.

8.8. Tool Abstraction (ADR-0054)

Namespace-gruppirovka vneshnih servisor: tool telegram { send(), get_updates() }. Vyzov cherez qualified call: telegram.send("id", "text"). Izoljacija namespaceov cherez qualified keys. 9 contract-testov.

8.9. Lifecycle Control (ADR-0056)

Checkpoint/resume dlja dolgih potokov: checkpoint("name") v flow. Serializacija sostojanija v SQLite ili in-memory. CLI: mlog resume. 10 contract-testov. Vosstanovlenie peremennyh i znachenij.

8.10. Security Audit (ADR-0057)

Statsicheskij bezopasnyj analiz: mlog audit. 8 proverok: SECRETS, SQL_DYNAMIC, SANDBOX_COVERAGE, RATE_LIMIT, CSRF, HTML_INJECTION, SECRET_LEAK, OPEN_REDIRECT. Taint tracking cherez peremennye. Exit codes: 0=clean, 1=errors, 2=warnings. 20/20 unit-testov.

9. Statistika i metriki

Metrika	Znachenie
Obshee chislo ADR	57 (iz nih 21 osnovnyh + 13 dopolnitelnyh naradov)
Fazy razrabotki	M1-M5 (core), Phase 1 (types), Phase 2 (advanced), Phase 3 (tooling), Phase 4 (VM)
Contract-testy	82/82 (vse projdeny, bez regressij)
Lib-testy	122/126 (4 predsystestvujushih oshibki v semantic)
Vozmizhnosti	Bytecode VM: 1.17-1.28x uskorenie nad tree-walk
Moduli Rust	parser, ast, interpreter, compiler, vm, audit, semantic, codegen, ir, embedding, llm, ml, builtins
Operatorov VM	30+ (vklyuchaja Metalogos-specific: Collapse, JumpIfLow, FlowExec)
Bezopasnye proverki audit	8 (secrets, SQL, sandbox, rate_limit, CSRF, HTML, leak, redirect)
Podderzhka koncepcij	entity, pattern, flow, rule, learnable, memory, fluid, tool, conversation

Tablica 2. Kljuchevye metriki Metalogosa na tekushij moment

10. Zakljuchenie

Vse 21 osnovnyh narada Metalogosa uspešno vypolneny i integrirovany v edinyj koda vuzovu. Jazyk proshel put ot prostogo konvejera entity-pattern-flow

do polnoj AML-platformy s bytecode VM, LSP-serverom, pakethymmedzherom, bezopasnym audito i upravlyaemym kontektom dialogov.

Kluchevye dostizheniya: (1) derevovidnyj interpreter i bytecode VM s polnoj paritetnostju, (2) trait-abstrakcii dlja LLM/ML backendov s mock i realnymi implementacijami, (3) Fluid Types s lenivym kollapsom i rasprostraneniem uverenosti, (4) bezopasnyj audit s taint-tracking, (5) 82 contract-testa bez regressij.

Dopolnitelnye narady (ADR-0045 - ADR-0057) rasshirajut osnovnyj funkcional dlja postroenija serioznych AML-prilozhenij: hooks dlja cross-cutting concerns, session memory dlja chat-botov, eval harness dlja CI/CD, conversation state dlja multi-turn dialogov, tool abstraction dlja integracii s vneshnimi servisa, checkpoint/resume dlja dolgih potokov i staticheskij bezopasnyj audit.

Dalnejshie napravlenija: peephole-optimizer dlja VM (Faza 4.3), realnye embeddings cherez PyO3 (Faza 2.3), remote registry dlja mlogpkg, incrementalnyj parsing dlja LSP, i primenenie vseh 21 fichej v Fosved office v2.